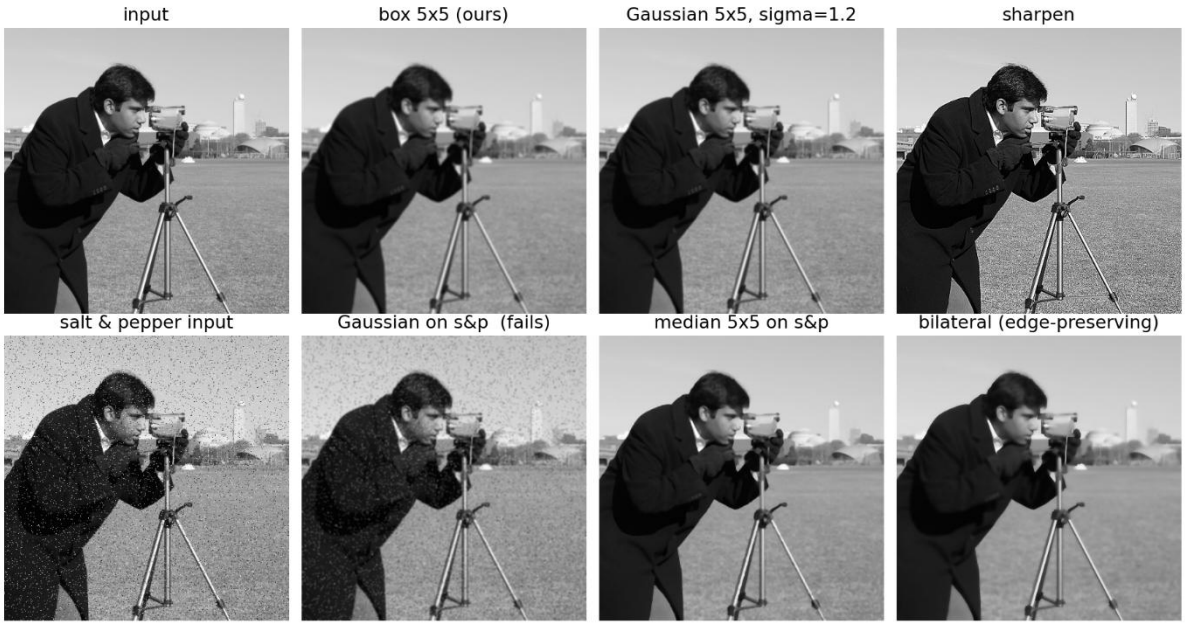
Digital Image Analysis Tutorial 2

```python
"""AI531C Tutorial T3 - Spatial Filtering and Convolution Studio (Tue 18 Aug 2026).
Runnable end-to-end; students edit only the marked TODO cells.
Inputs : inputs/camera.png, inputs/camera_saltpepper.png (created if absent)
Outputs: expected_outputs/T03_verify.txt, T03_filters_panel.png, T03_param_sweep.png
"""
import numpy as np, cv2, os
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from skimage import data

HERE = os.path.dirname(os.path.abspath(__file__))
T03  = os.path.join(HERE)
INP, OUT = os.path.join(T03, "inputs"), os.path.join(T03, "expected_outputs")
os.makedirs(INP, exist_ok=True); os.makedirs(OUT, exist_ok=True)

# ---------- 0. Inputs (offline-safe) ----------
cam_p = os.path.join(INP, "camera.png")
sp_p  = os.path.join(INP, "camera_saltpepper.png")
if not os.path.exists(cam_p):
    cv2.imwrite(cam_p, data.camera())
if not os.path.exists(sp_p):
    img = data.camera().copy(); rng = np.random.default_rng(0)
    m = rng.random(img.shape); img[m < 0.025] = 0; img[m > 0.975] = 255
    cv2.imwrite(sp_p, img)
cam = cv2.imread(cam_p, 0); sp = cv2.imread(sp_p, 0)

# ---------- 1. PART A: convolution from scratch ----------
def conv2d(f, k):
    """True 2-D convolution (kernel flipped), reflect padding, same size."""
    k = np.flipud(np.fliplr(k)).astype(float)
    P = k.shape[0] // 2
    fp = np.pad(f.astype(float), P, mode="reflect")
    out = np.zeros(f.shape, float)
    for i in range(k.shape[0]):            # loop over KERNEL, not pixels
        for j in range(k.shape[1]):
            out += k[i, j] * fp[i:i + f.shape[0], j:j + f.shape[1]]
    return out

# verify on the L08 lecture patch (hand-checked in class: centre values -2 and 11)
F4 = np.array([[1,2,1,0],[0,1,3,1],[2,2,1,0],[1,0,0,2]], float)
K_sharp = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]], float)
mine   = conv2d(F4, K_sharp)
opencv = cv2.filter2D(F4, -1, K_sharp, borderType=cv2.BORDER_REFLECT_101)  # matches np.pad mode="reflect"
with open(os.path.join(OUT, "T03_verify.txt"), "w") as fh:
    fh.write("conv2d (ours):\n%s\n\ncv2.filter2D:\n%s\n\nmax |diff| = %.6f\n"
             % (mine, opencv, np.abs(mine - opencv).max()))
    fh.write("\nNote: filter2D computes CORRELATION. It equals our convolution here\n"
             "because this kernel is symmetric. Padding note: np.pad(reflect) ==\n"
             "cv2.BORDER_REFLECT_101 (edge not repeated). TODO(discussion): find a\n"
             "kernel where correlation and convolution disagree.\n")

# ---------- 2. PART B: the filter zoo on real inputs ----------
K_box = np.ones((5, 5)) / 25.0
panels = [
    (cam, "input"),
    (conv2d(cam, K_box), "box 5x5 (ours)"),
    (cv2.GaussianBlur(cam, (5, 5), 1.2), "Gaussian 5x5, sigma=1.2"),
    (cv2.filter2D(cam, -1, K_sharp), "sharpen"),
    (sp, "salt & pepper input"),
    (cv2.GaussianBlur(sp, (5, 5), 1.2), "Gaussian on s&p  (fails)"),
    (cv2.medianBlur(sp, 5), "median 5x5 on s&p"),
    (cv2.bilateralFilter(cam, 9, 60, 10), "bilateral (edge-preserving)"),
]
fig, ax = plt.subplots(2, 4, figsize=(14, 7.4))
for a, (im, t) in zip(ax.ravel(), panels):
    a.imshow(np.clip(im, 0, 255), cmap="gray", vmin=0, vmax=255)
    a.set_title(t, fontsize=15); a.axis("off")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "T03_filters_panel.png"), dpi=130); plt.close()

# ---------- 3. PART C: parameter sweep ----------
sizes = [3, 7, 15]
```

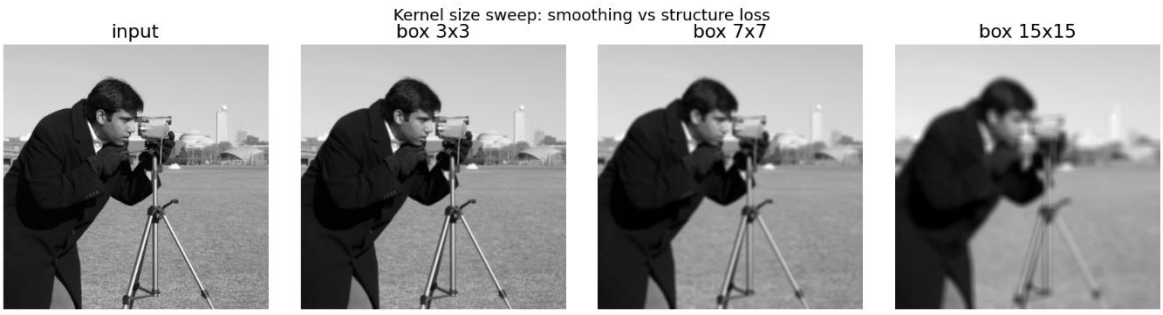
```python
fig, ax = plt.subplots(1, 4, figsize=(13.5, 3.6))
ax[0].imshow(cam, cmap="gray"); ax[0].set_title("input", fontsize=15); ax[0].axis("off")
for a, n in zip(ax[1:], sizes):
    a.imshow(cv2.blur(cam, (n, n)), cmap="gray")
    a.set_title(f"box {n}x{n}", fontsize=15); a.axis("off")
plt.suptitle("Kernel size sweep: smoothing vs structure loss", fontsize=13)
plt.tight_layout(); plt.savefig(os.path.join(OUT, "T03_param_sweep.png"), dpi=130); plt.close()

print("T03 studio complete. Verify file and panels written to expected_outputs/.")
print("max |ours - filter2D| on test patch:", np.abs(mine - opencv).max())
```

Output Images:

Filters Panel



Parameter Sweep



conv2d (ours):

[[1. 6. -3. -4.]

[-5. -2. 11. -1.]

[5. 6. 0. -5.]

[1. -5. -4. 10.]]

cv2.filter2D:

[[1. 6. -3. -4.]

[-5. -2. 11. -1.]

[5. 6. 0. -5.]

[1. -5. -4. 10.]]

max |diff| = 0.000000